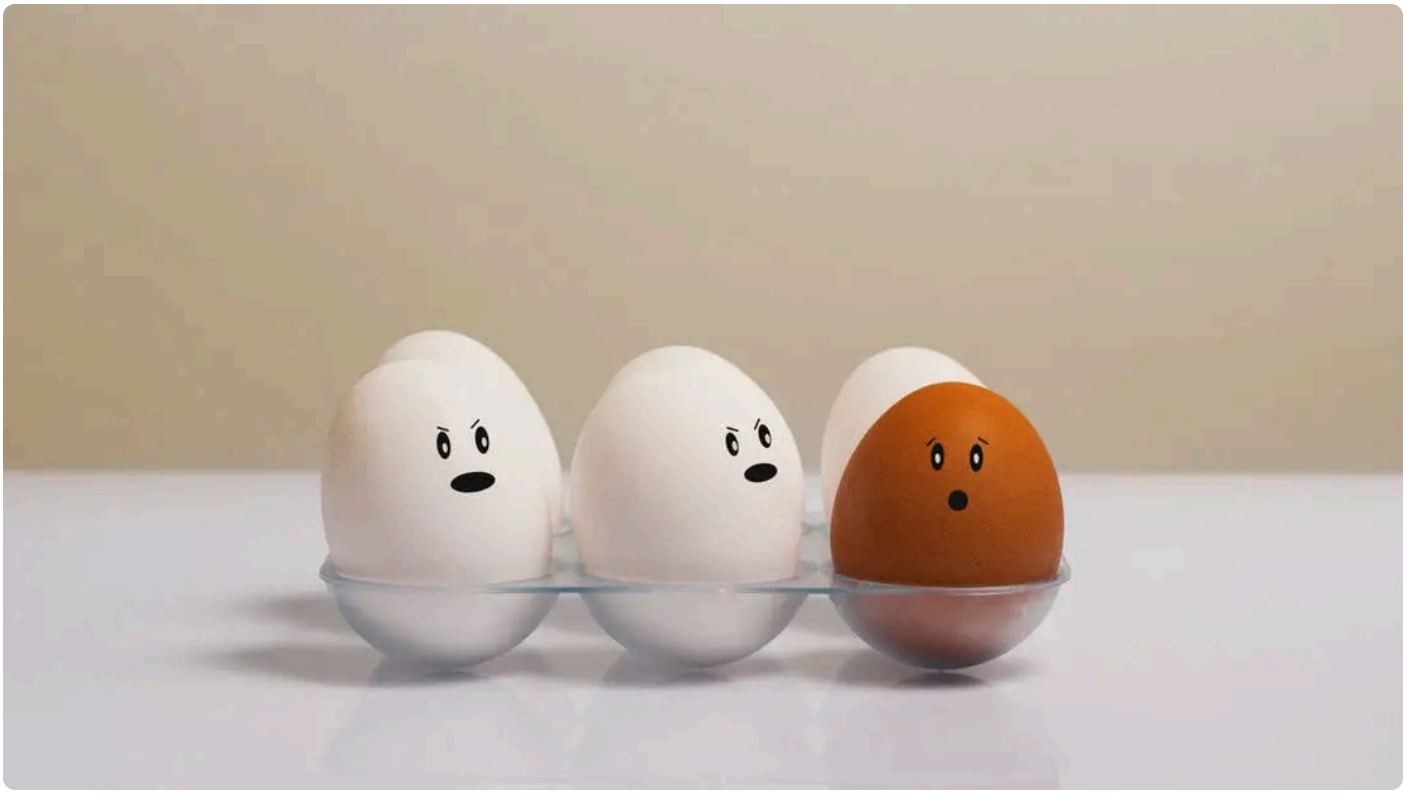
前沿技术速递｜Agent Teams：打造你的第一支“虚拟研发团队”

Tony Bai · AI 原生开发工作流实战



你好，我是 Tony Bai。

时光飞逝，转眼我们已经来到了 2026 年。

回顾过去的一年，如果说 2025 年是 Coding Agent 的元年，以 Claude Code 为代表的工具让“自然语言编程”真正成为了现实；那么 2026 年，我们正站在一个新的风口之上——Agent Orchestration（智能体编排）。

在专栏之前的课程中，我们已经精通了如何与单个 Claude Code 实例进行深度协作，甚至学会了用 Sub-agent 让它在同一个会话中“切换人格”。但这仍然是一个“单兵作战”的模式。随着任务复杂度的提升，单个 Agent 的上下文窗口（Context Window）和注意力很容易成为瓶颈。

我们不禁会想：如果能同时召唤多个 Claude Code，让它们像一个真实的开发团队那样，有分工、有协作、甚至互相 Code Review，会是怎样的景象?

Claude Code 最近在 2.1.32 版本中发布的重磅实验特性——**Agent Teams（智能体团队）**，正是对这一未来的提前兑现。

今天这篇加餐，我就带你深入了解这个代表未来的新特性。我们将跳出“人机结对”的框架，尝试扮演一次“技术总监”，指挥一支由 AI 组成的“虚拟研发团队”，去并行解决一个全栈开发问题。

Agent Teams：从“单兵”到“集群”的范式跃迁

首先，我们需要厘清 Agent Teams 与我们之前学过的 Sub-agents（第 14 讲）有何本质区别。

Sub-agents（智能分身）是在同一个会话中，通过加载不同的 System Prompt 来切换 AI 的角色，通常执行专注于结果导向的任务。虽然不会“继承”主 Agent 的上下文，不会因为主 Agent 的上下文过长干扰 Sub-agent 的工作任务，但一个 Claude Code 会话一次只能执行一个 Sub-agent，是串行的。在项目规模扩大时，这种模式的任务执行或问题解决效率要低得多。

而 Agent Teams 采用了“Team Lead + Teammates”的架构：

> Team Lead（队长）：就是你当前交互的那个 Claude 会话。它负责拆解任务、分配工作、协调进度、合成结果。它拥有全局视野。
>
> Teammates（队员）：是完全独立的 Claude Code 实例，由 Lead 动态生成和管理。

Agent Teams（智能体团队）本质上是启动多个独立的 Claude Code 进程。它具有如下优势：

1. 完全并行：前端、后端、测试可以同时开工，互不阻塞。
2. 独立上下文：每个队员（Teammate）都有自己独立的上下文窗口，只关注自己的任务，不会被其他队员的琐碎细节污染。
3. 自主协同：队员之间可以通过“消息 (mailbox)”和“共享任务列表（Shared Task List）”进行直接沟通，无需人类充当传声筒。

业界前沿：16 个 Claude 重写 C 编译器的故事

为了验证这种“集群协同”模式的极限，Anthropic 的研究团队做了一个疯狂的实验：让 Agent Teams 从零开始使用 Rust 语言写一个 C 编译器。这个任务极其复杂，涉及词法分析、语法分析、IR 生成、寄存器分配、汇编生成等多个深奥领域。单靠一个 Context 窗口绝对无法容纳所有细节。

他们是这么做的：

1. 规模：启动了 16 个 Claude Code Agent 并行工作。
2. 协作模式：基于 Git 仓库进行同步。每个 Agent 领取任务后，在本地修改、测试，然后推送到上游，解决合并冲突。
3. 精细分工：有的负责实现具体功能（如“实现 if 语句解析”），有的负责“代码去重”，有的负责性能优化，甚至有的专门负责扮演“Rust 专家”来对代码架构进行批评和指正。
4. 测试驱动：引入了 GCC 作为“预言机”（Oracle），对比 Claude 编译器的输出和 GCC 的输出，确保正确性。

结果如何呢?

在消耗了约 20 亿输入 Token 后，这个 Agent 团队写出了一个 10 万行代码的编译器。它不仅能编译 SQLite、Redis、PostgreSQL 的系统软件或中间件，甚至可以在 x86、ARM、RISC-V 架构上成功编译 Linux 6.9 内核。

当然，这个由 Agent 团队实现的 C 编译器还有这样那样的问题，比如生成的代码的执行效率不高、没有自己的汇编器和链接器（借助 GCC）、还不能真正替代真正的 GCC 编译器等。但这个案例向我们证明了：当多个 Agent 拥有独立上下文并能协同工作时，它们能解决的工程复杂度将呈指数级上升。

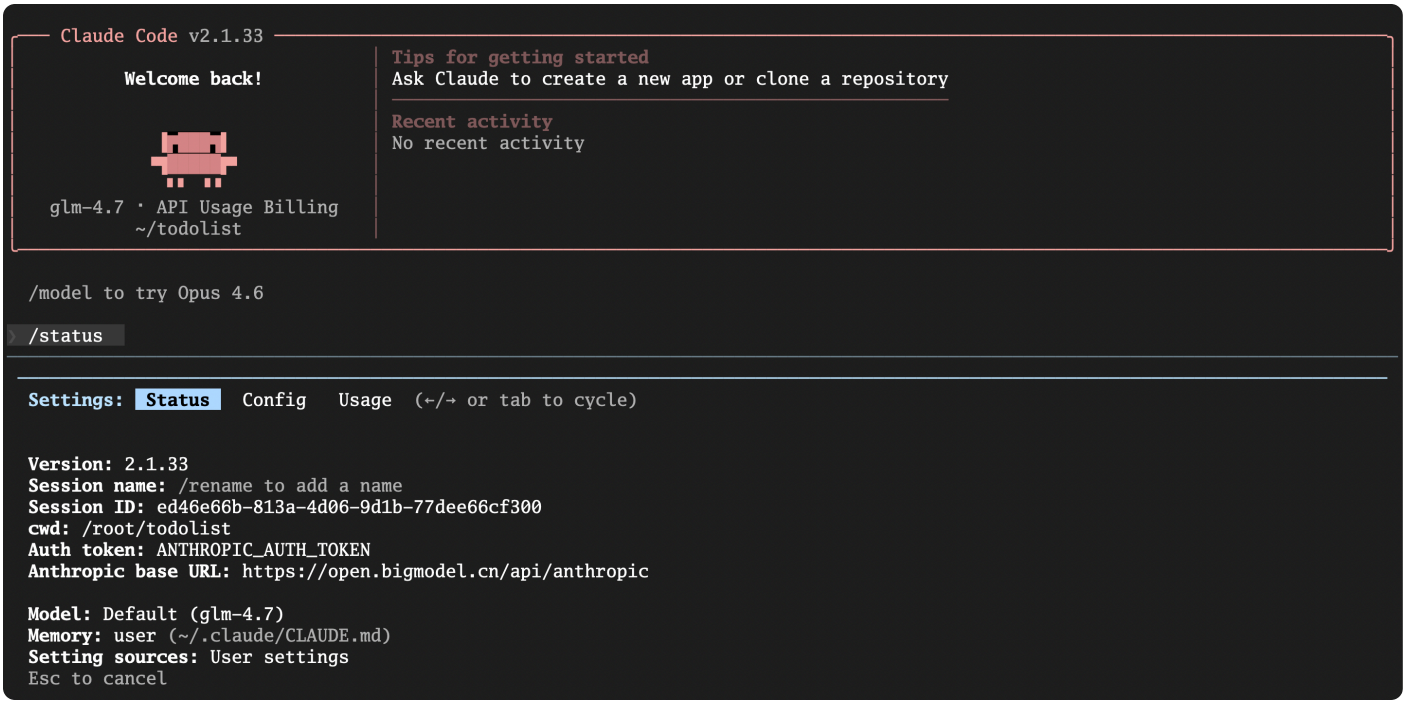
实战演练：开发一个“待办事项（Todo）”全栈应用

C 编译器太复杂了，为了让你能直观地体验 Agent Teams 的威力，我们来做一个更接地气的实验：在 10 分钟内，开发一个包含前端、后端和数据库的 Todo List 应用。我们将组建一支

由三个 AI 专家组成的“虚拟研发团队”。

1. 环境准备与开启特性

Agent Teams 目前（截至 2026 年 2 月初）在 Claude Code 中仍属于实验特性（Experimental），且需要较新的版本（2.1.32 及以后版本）支持。在真正落地使用时，具体表现可能与本讲描述略有出入。你可以启动 claude 后，使用`/status`命令查看你的 Claude Code 版本，就像下图这样：



要使用 Agent Teams，你首先需要显式开启它：

复制代码

```
# 在 ~/.claude/settings.json 设置

{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}

# 或环境变量中设置
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

建议使用 iTerm2 (macOS) 或 tmux 环境，因为 Agent Teams 支持 Split Panes（分屏显示）模式，能让你同时看到所有 Agent 的工作状态，那是真正的“上帝视角”。

不过限于开发环境，本讲无法给大家展示这么炫酷的效果了。我们采用的是“in-process”的模式，即所有 teammate agent 都在一个会话里展示自己的工作过程与结果。Claude Code 默认“teammateMode”设置是 auto，即它会根据运行环境自动判定使用“分屏显示模式”，还是“进程内显示模式”。

2. 下达指挥官指令

在一个空目录 `todolist` 下，我们启动 Claude Code，并输入以下 Prompt：

> 我要开发一个最简单的 Todo List 应用，包含：
> 后端：使用 Go + Gin + SQLite，提供 RESTful API（增删改查）。
> 前端：使用 React + Vite + TailwindCSS，实现单页应用。
> 测试：为后端 API 编写集成测试。
> 请创建一个 Agent Team 来并行完成这个任务：
> Backend Dev：负责后端代码和数据库设计。
> Frontend Dev：负责前端页面和 API 调用。
> QA Engineer：负责编写测试脚本。
> 你是 Team Lead。请先规划 API 接口定义，然后指挥前后端并行开发，最后由 QA 进行验证。**

3. 观察 Team Agent 间的协作

此时，Claude 会化身为 **Team Lead**。它会分析你的请求，并初始化团队，如下图所示：

● 我来帮你规划并创建这个Todo List项目的开发团队。首先创建团队和任务列表，然后启动并行开发。

● 团队已创建。现在让我先规划API接口定义，然后创建任务列表。

● **Write**(API_DESIGN.md)
  └ Wrote **164** lines to **API_DESIGN.md**
```
     1 # Todo List API Design
     2
     3 ## Base URL
     4 ```
     5 http://localhost:8080/api/v1
     6 ```
     7
     8 ## Data Models
     9
    10 ### Todo Item
   … +154 lines (ctrl+o to expand)
```

● API设计已完成。现在创建任务列表并启动团队成员：

● 任务列表已创建。现在启动团队成员并行开发：

● **3** agents launched (ctrl+o to expand)
  ├─ **@backend-dev**
  │ └ Backend Dev: Go/Gin/SQLite (shift+↑ to manage)
  ├─ **@frontend-dev**
  │ └ Frontend Dev: React/Vite/Tailwind (shift+↑ to manage)
  └─ **@qa-engineer**
     └ QA Engineer: Integration testing (shift+↑ to manage)

● 团队已组建完成！让我向团队成员发送启动指令：

● **Team 已组建完成！** 🚀

团队成员

成员	角色	Agent ID
Team Lead	项目协调	team-lead@todolist-dev
backend-dev	Go/Gin/SQLite开发	backend-dev@todolist-dev
frontend-dev	React/Vite开发	frontend-dev@todolist-dev
qa-engineer	测试工程师	qa-engineer@todolist-dev

任务列表

ID	任务	状态	负责人
1	Setup Go backend structure	Pending	backend-dev
2	Implement Todo CRUD API	Pending	(blocked by #1)
3	Create React + Vite project	Pending	frontend-dev
4	Implement Todo UI components	Pending	(blocked by #3)
5	Write API integration tests	Pending	qa-engineer

API设计文档 ✅

已生成 /root/todolist/API_DESIGN.md，包含：
- 6个RESTful端点定义
- 数据模型和数据库Schema
- 请求/响应格式

团队成员已收到启动指令并开始工作。我会监控任务进度，请稍候...

接下来，各个 Teammate 开始领任务并开始工作，以 qa-engineer 为例，我们看到它的过程输出：

```
@qa-engineer❯ QA Engineer: Waiting for backend implementation

● 收到 QA Engineer 的状态报告。让我检查当前任务进展：

● 看到 backend-dev 正在处理 Task 9（与 Task 1 相同）。让我更新任务分配并设置依赖关系：

● 当前任务状态

  ┌────┬──────────────────────────────┬──────────────┬──────────────┬──────┐
  │ ID │ 任务                         │ 状态         │ 负责人       │ 阻塞 │
  ├────┼──────────────────────────────┼──────────────┼──────────────┼──────┤
  │ 1  │ Setup Go backend structure   │ 🔄 IN_PROGRESS │ backend-dev  │ -    │
  │ 2  │ Implement Todo CRUD API      │ ⏸ PENDING    │ backend-dev  │ #1   │
  │ 3  │ Create React + Vite project  │ 🔄 IN_PROGRESS │ frontend-dev │ -    │
  │ 4  │ Implement Todo UI components │ ⏸ PENDING    │ frontend-dev │ #3   │
  │ 5  │ Write API integration tests  │ ⏸ PENDING    │ qa-engineer  │ #2   │
  └────┴──────────────────────────────┴──────────────┴──────────────┴──────┘

  QA Engineer 已确认待命，等待后端API完成。

@qa-engineer❯ QA Engineer: Task 5 认领确认

@qa-engineer❯ Task 5 blocked by backend
```

如果任务被其他 teammate 的任务阻塞，那么该 agent 会等待。就像 qa-engineer 要等待 backend-dev 和 frontend-dev 完成一些开发任务后才能开始执行测试。

Team lead 会定期收集各个 Teammate 的工作进展：

● 看到项目结构已初步搭建。让我向团队了解进展：

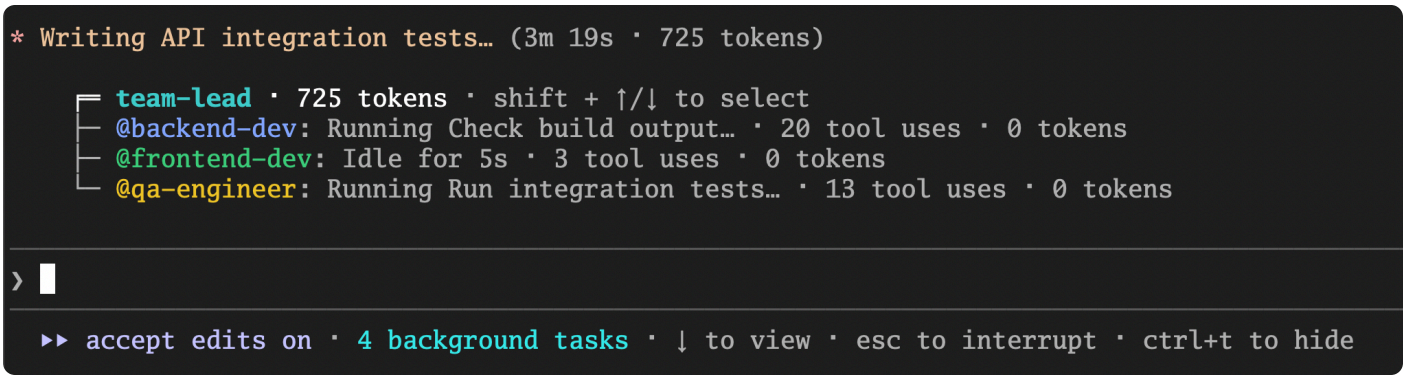
● 进度同步

当前项目状态：

组件	状态
API Design	✅ 完成
Backend Models	✅ todo.go
Backend Database	✅ db.go
Backend Handlers	⏸ 待创建
Backend Main	⏸ 待创建
Frontend Vite	✅ 配置完成
Frontend Components	🔄 AddTodoForm 已创建

已向 backend-dev 和 frontend-dev 请求进度汇报。等待响应...

作为人类指挥官，你可以随时介入，查看各个 teammate 的当前状态（使用 ctrl+t 打开 teammate 列表）：



> 注：teammate 的初始权限设置与 team lead 相同。如果 lead 权限设置为 `--dangerously-skip-permissions`，则所有队友的权限也设置为 `--dangerously-`

> `skip-permissions`。使用`--dangerously-skip-permissions`权限会让 team 工作更流畅，否则需要人工反复确认各个 Teammate agent 的 执行权限。

4. 验收与收尾

当所有任务都完成后，Lead 会向各个 Teammate 确认。

Frontend-dev 确认已经关闭：

● 所有任务已完成。等待其他团队成员确认关闭...

团队成员状态：
- frontend-dev: ✅ 已关闭
- backend-dev: ⏳ 等待确认
- qa-engineer: ⏳ 等待确认

📦 **项目交付摘要**

Todo List 全栈应用 已完成开发：

层级	技术栈	文件数
后端	Go + Gin + SQLite	5 个 .go 文件
前端	React + Vite + TailwindCSS	7 个 .jsx/.js 文件
测试	Go testing	1 个测试套件 (745 行)
文档	Markdown API 设计	1 个

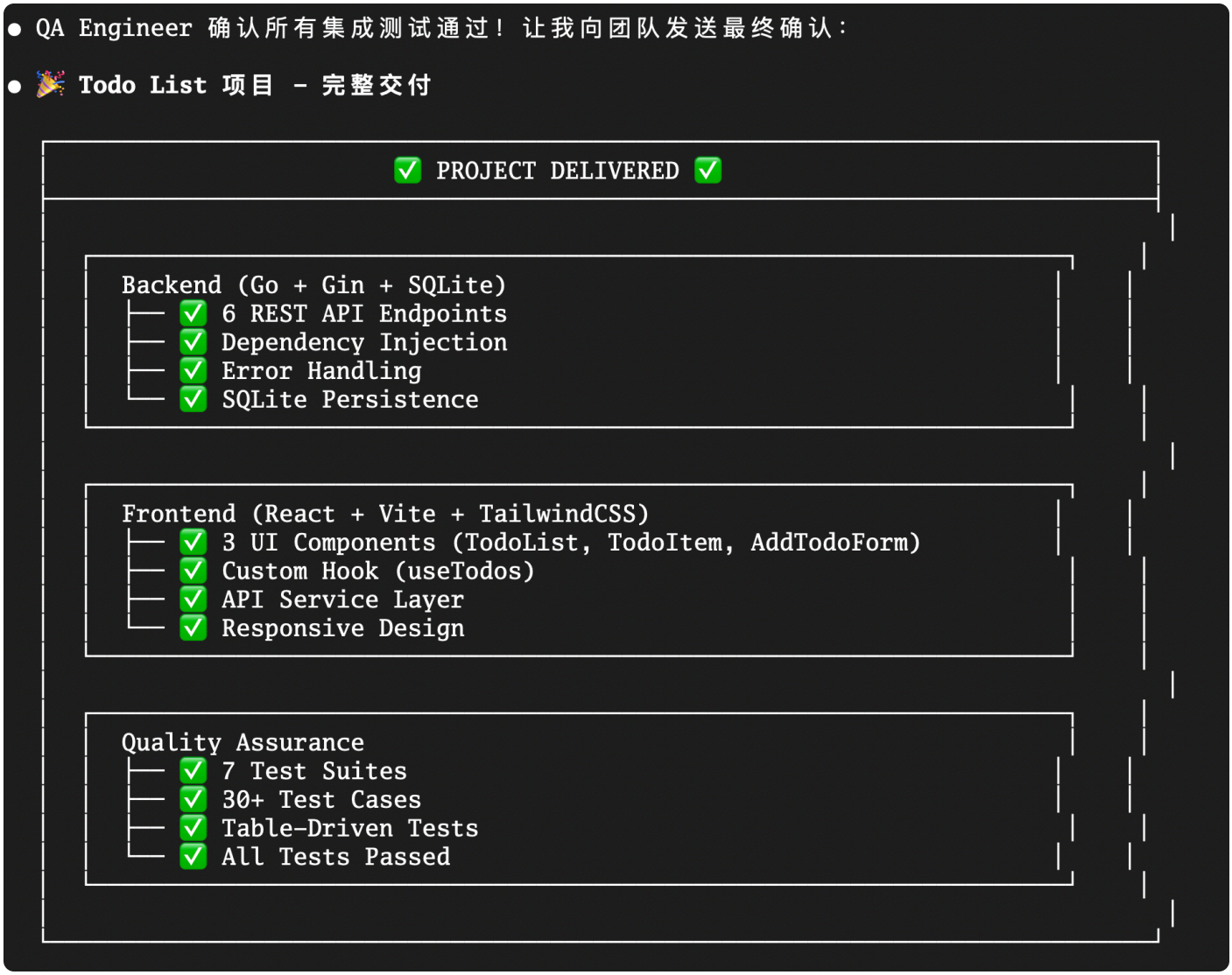
启动命令：

```
# 后端
go run cmd/server/main.go

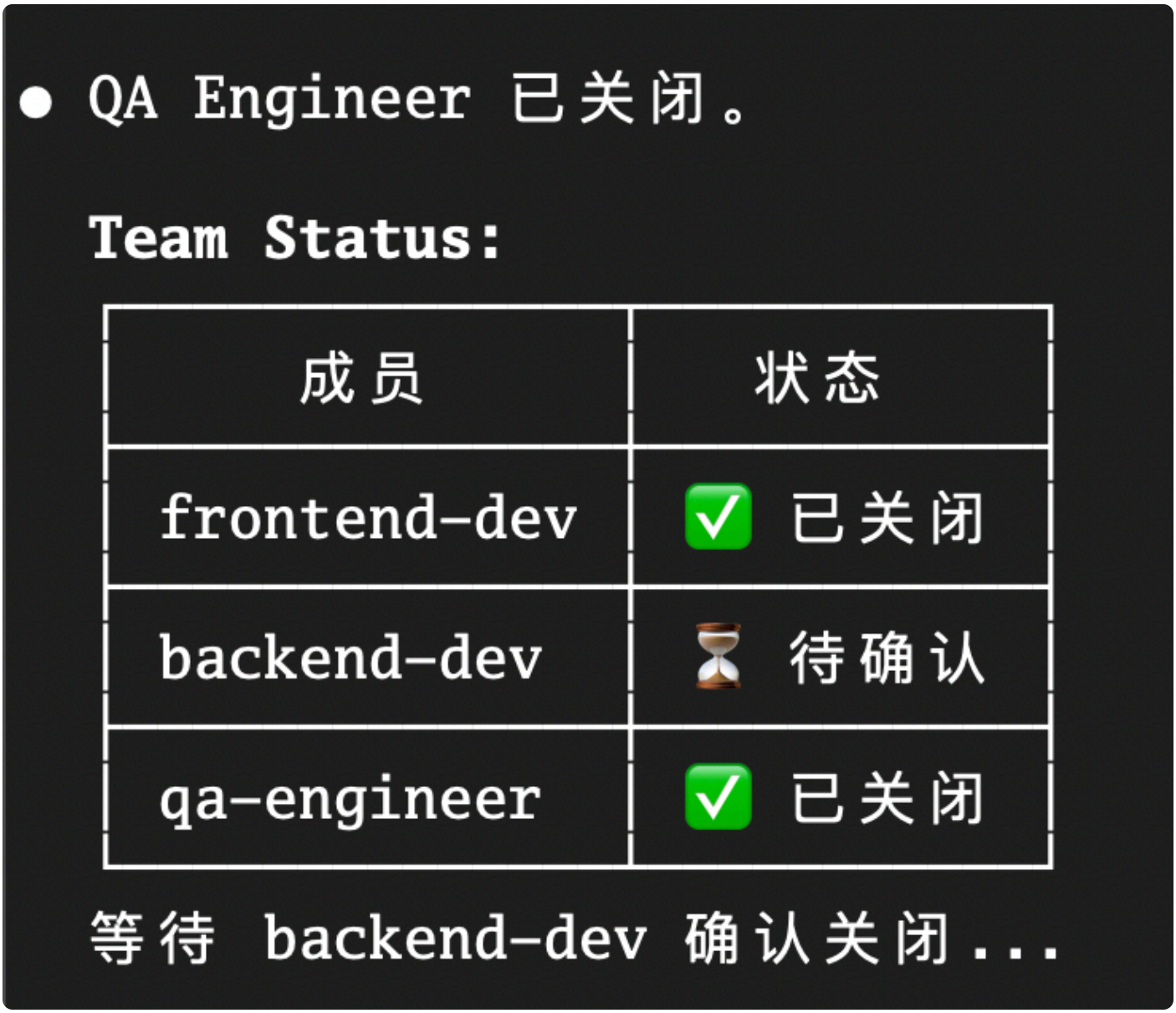
# 前端 (需要 Node.js)
npm install && npm run dev

# 测试
go test ./tests/integration/... -v
```

Qa-engineer 确认已经完成所有测试：



Qa-engineer 确认已经关闭：



最后等待 Backend-dev 确认关闭：

```
● Teammate @qa-engineer shut down gracefully
● 任务列表已清空 - 所有工作已完成！
  等待 backend-dev 确认关闭后将解散团队。

@backend-dev› Backend API implementation completed
* Crunched for 38m 29s
● Backend Dev 确认完成！
● Teammate @backend-dev shut down gracefully
● Backend Dev 任务已完成。让我检查团队最终状态：
```

当所有任务完成且所有 Teammate 都确认关闭后，Team Lead 会清理团队并向你汇报：

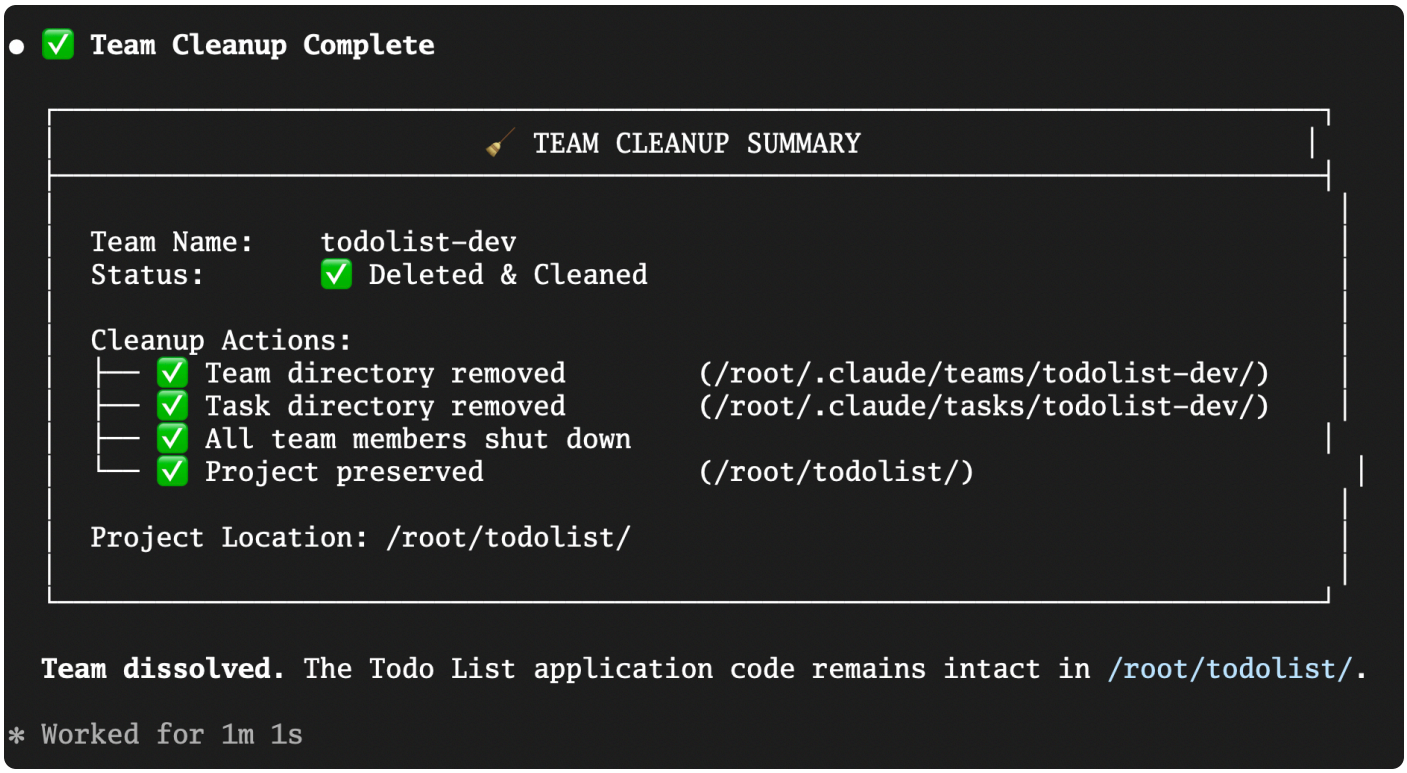
```
● 所有团队成员已完成工作并退出。现在可以清理团队资源：

● 团队已解散。

  ---
  🎉 Todo List 应用 - 最终交付报告

  ┌──────────────────────────────────────────────────────────────────────┐
  │                                                                      │
  │                        ✅ 项目交付完成 ✅                            │
  │                                                                      │
  │  技术栈                                                              │
  │  ├── Backend:  Go 1.25 + Gin + SQLite                                │
  │  ├── Frontend: React 18 + Vite + TailwindCSS                         │
  │  └── Tests:    Go testing framework (Table-Driven Tests)             │
  │                                                                      │
  │  项目统计                                                            │
  │  ├── Go 源文件：      5 个 (.go)                                     │
  │  ├── 前端文件：       7 个 (.jsx/.js)                                │
  │  ├── 测试用例：       30+ 个                                         │
  │  └── 总代码行数：     ~1500+ 行                                      │
  │                                                                      │
  │  API 端点 (6个)                                                      │
  │  ├── GET    /api/v1/todos           列出所有 todos                   │
  │  ├── POST   /api/v1/todos           创建新 todo                      │
  │  ├── GET    /api/v1/todos/:id       获取单个 todo                    │
  │  ├── PUT    /api/v1/todos/:id       更新 todo                        │
  │  ├── DELETE /api/v1/todos/:id       删除 todo                        │
  │  └── PATCH  /api/v1/todos/:id/toggle 切换完成状态                    │
  │                                                                      │
  │  启动命令                                                            │
  │  ├── 后端：  go run cmd/server/main.go    (端口 8080)                │
  │  ├── 前端：  npm install && npm run dev    (端口 3000)               │
  │  └── 测试：  go test ./tests/integration/... -v                      │
  │                                                                      │
  └──────────────────────────────────────────────────────────────────────┘
```

这里 team 的工作痕迹可能依然会存留在你的环境里，你可以显式输入 `Clean up the team`，清除这些“痕迹”，并确认团队都已经清理完毕：



从图片中可以看到，Agent Team 工作时，团队配置信息和 task 信息都存储在本地：

团队配置一般放在：

```
~/.claude/teams/{team-name}/config.json
```

而 task 配置放在：

```
~/.claude/tasks/{team-name}/
```

最佳实践与注意事项

虽然 Agent Teams 很强大，但它也带来了新的复杂性和成本。以下是几条从实践中总结的经验：

1. **任务粒度要合适**

   a. 不要为了用而用。如果任务是线性的（如“先改 A 文件，再改 B 文件”），单个 Agent 足够了，甚至更快。

   b. 并行度高、模块隔离度高的任务（如前后端联调、多语言翻译、微服务拆分）才是 Agent Teams 的最佳舞台。

2. **关注 Token 成本**

   a. 请记住，每个 Teammate 都是一个独立的 Claude 实例，拥有独立的上下文。启动一个 5 人的团队，Token 消耗速度可能是平时的 5 倍。

   b. 善用委派模式（Delegate Mode）：让 Team Lead 只负责指挥和传递消息，不自己写代码，防止它“抢活”导致上下文浪费。创建一个团队后，按 Shift+Tab 切换到委派模式。

3. **明确的角色定义**

   a. 在创建 Team 时，给每个 Teammate 明确的角色定义（System Prompt）。你是“最苛刻的安全审计员”，你是“追求极致性能的 C++ 专家”。角色越鲜明，协作效果越好。

4. **显示模式**

   a. 如果环境满足，推荐使用 Split Panes (tmux/iTerm2) 模式。能同时看到所有 Teammate Agent 的“思考过程”，不仅便于监控，这种“赛博朋克”风格的体验本身就极具未来感。如果环境允许，大家可以自行体验。

本讲小结

Agent Teams 的出现，标志着我们从“人机结对编程”迈向了“人机组织管理”。

首先，并行执行和自主协同是 Agent Teams 区别于 Sub-agents 的核心优势，解决了复杂任务中规划分工和效率瓶颈的问题。

其次，通过 Team Lead + Teammates 的架构，我们可以在本地模拟出一个微型的“软件开发团队”，实现从需求到交付的全栈自动化。成本与收益的权衡是使用该特性的关键。它适合解决那些单兵难以应付的、结构复杂的工程难题。

最后，在不久的将来，评价一个高级工程师的标准，可能不再仅仅是你自己能写多少代码，而是你能否指挥一支由 AI 组成的“虚拟研发团队”，在半小时内解决过去需要一周才能完成的复杂工程问题。

希望这篇加餐能让你提前感受到 2026 年软件工程的脉搏。虽然它现在还是实验特性，但**未来已来，只是分布尚不均匀**。去试试吧，组建你的第一支 AI 战队！

思考题

如果让你现在就组建一支 Agent Team 来辅助你的日常工作，你会如何设计你的“队员”配置？

> 你需要一个专门负责 Code Review 的队员吗？
>
> 你需要一个专门负责写文档的队员吗？
>
> 还是需要一个专门负责紧盯错误日志的运维队员？

请在评论区分享你的“AI 虚拟研发团队”配置方案，让我们看看谁的团队设计最高效！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (17)

最新　精选



流浪地球

2026-03-03 来自广东

看到了K8S的影子，智能体编排也是个大热方向了

> 作者回复: 👍

9

Geek_f00be9
2026-03-24 来自浙江

我看cc官方说teammates之间可以相互通信，没有看出来咋通信的

作者回复: 通信这块具体实现方式是“黑盒”。目前看主要聚焦在team的成员之间的任务派发、相互进度查询或通知啥的。

3

这次多学点
2026-02-14 来自浙江

上周同看到agent teams，被他的这个PR效果震惊，2周开发一个基本可用的C编译器，人工专家也很难做到了

作者回复: 是的。agent编排是2026的重点和亮点

3

Geek_be37e5
2026-02-08 来自上海

老师好，请问Agent Team解决的主要问题是SubAgent不能并行执行的问题嘛?

作者回复: 不完全是。

SubAgent (单兵)的重点是“专业性”和“上下文隔离”。我派一个专家去钻研一个“深坑”，不干扰主线。

Agent Team (团战)的重点是“协作”和“复杂任务拆解”。它解决的是一个任务太复杂，需要多个角色（产品、开发、测试）互相配合、多轮交互才能完成的问题。并行只是其中的一种调度策略，更重要的是分工与协同。

3

黄阳
2026-04-03 来自福建

agent teams如何和sdd结合起来，后面sdd的演示过来更像需要更多人为的参与，但是agent teams更多的不需要人为参与

作者回复: 我们专栏中演示的SDD流程，确实在每个“编译”阶段（specify –> plan –> tasks）都保留了人类审批的关键节点，这是一种高可靠、高可控的实践模式。而Agent Teams则追求在Leader的指挥下，实现最大程度的并行与自主。

不过并不矛盾。CC的 Teams可以作为SDD流程的“执行引擎”。

但如果任由一个Agent Team自由发挥，它们可能会因为缺乏统一目标而陷入混乱，尤其是大型项目。而spec.md, plan.md, tasks.md这一套SDD产物，可成为整个Team共享的、唯一的需求来源以及路线图。

SDD负责顶层的“战略规划”，而Agent Teams负责底层的“战术执行”。

当然针对一些简单的小工程，也可以直接像文中示例那样，直接让team中的Team Lead来分析和拆解task，并交于team mate去完成。

2



H

2026-09-10 来自广东

老师您好，关于文中这段：

“一个 Claude Code 会话一次只能执行一个 Sub-agent，是串行的。在项目规模扩大时，这种模式的任务执行或问题解决效率要低得多。”

想请教一下，这个描述是否已经和目前 Claude Code 的实际能力存在一些版本差异?

我查到 Anthropic 当前官方文档对 Subagent 的使用建议中明确提到：

“Use subagents when tasks can run in parallel, require isolated context, or involve independent workstreams that don’t need to share state.”

从这段官方描述来看，Subagent 目前应该已经支持用于可并行执行的独立任务，因此“一个会话一次只能执行一个 Sub-agent，是串行的”这个表述，似乎不再适合作为当前能力的绝对结论。

我理解 Agent Teams 相比 Subagent 的核心优势，可能更多在于独立实例、持续协作、共享任务管理和成员间通信能力，而不只是“Subagent 串行、Agent Teams 并行”的区别。

不知道这样理解是否准确，也想请老师确认一下这里是否需要结合当前版本更新表述。

作者回复: 你说的没错，这篇文章是team mode刚出来的时候写的。claude code演进很快，建议大家结合最新claude code的特性来理解。






AionX
2026-05-04 来自湖北

太强了，赶快学起来。

作者回复: [手动抱拳]






奕
2026-03-10 来自美国

这个功能厉害

作者回复: 嗯，team值得重点研究和实践。






友
2026-03-08 来自湖南

对于拆分我能想到 前端开一个 后端开一个 产品开一个。 但是如果要更细节 我就怕分的不够好

作者回复: 先尝试，看效果，再优化。






许凯
2026-02-13 来自江苏

后面估计还会出整体调度安装在不同机器上claude code Agent的功能吧

作者回复: 👍 终极软件工厂也会在不远的将来出现😄



